DATA-260 Homework 8 – Report

Uday Patel

GitHub repository: `https://github.com/UdayPate/data260-9871`

Section 0 – Personal Configuration and Domain

Value	Result
SID4	9871
PORT_BASE	8871
PREFIX	s9871
SEED	9871
VERIFY_SEED	269871
DOMAIN_ID	7 – Community sports league fixtures

Hardware: Lenovo Legion Slim 5 16" (Ryzen 5 7640HS, 16GB RAM, RTX 4060 8GB, 512GB SSD).
Local model used: qwen3:8b via Ollama (thinking off, sampling seed 9871) in Part 2 only. Part 1 uses no model. The Part 3 failure tests run the Writer in a model-free stub mode (disclosed in Part 3).
Kafka: official apache/kafka:3.9.0 image, single node (KRaft), docker compose; Python client confluent-kafka 2.16.1; FastAPI on port 8871.
Repository: `https://github.com/UdayPate/data260-9871`
Tagged commit: PLACEHOLDER_HASH (to be filled in after the final commit and hw8 tag)
`https://github.com/UdayPate/data260-9871/tree/PLACEHOLDER_HASH`
Collaborator access: confirmed working for Sbnikitha and supriyaselvanganesan.

Part 1 – Backend Service Request/Response over Kafka

The Fixture service (domain 7) is split into a client side (FastAPI wrapper `api.py`, the demo `homework_demo.py`, both using `kafka_rpc.KafkaRpcClient`) and a worker (`worker.py`) that talk only through two Kafka topics. The worker keeps the fixtures in a plain Python dict and generates `entity_id` with `uuid4`. Kafka client: **confluent-kafka** (Confluent's own, actively released librdkafka client with Windows and Linux wheels, manual offset commits and partition control).

Topics and docker compose

Topic	Written by	Read by	Partitions
`s9871_entity_requests`	client / FastAPI	worker	1
`s9871_entity_responses`	worker	client / FastAPI	1

`docker compose up` starts the official `apache/kafka:3.9.0` image in single-node mode, a one-shot `kafka-init` job that creates the topics, the worker and the FastAPI service. Excerpt (broker service):

```
kafka:
  image: apache/kafka:3.9.0
```

```
    container_name: s9871-kafka
    ports:
      - "9092:9092"
    environment:
      KAFKA_NODE_ID: 1
      KAFKA_PROCESS_ROLES: broker,controller
      KAFKA_CONTROLLER_QUORUM_VOTERS: 1@kafka:9093
      KAFKA_LISTENERS: INTERNAL://:19092,EXTERNAL://:9092,CONTROLLER://:9093
      KAFKA_ADVERTISED_LISTENERS: INTERNAL://kafka:19092,EXTERNAL://localhost:9092
      KAFKA_LISTENER_SECURITY_PROTOCOL_MAP: INTERNAL:PLAINTEXT,EXTERNAL:PLAINTEXT,CONTROLLER:PLAINTEXT
      KAFKA_INTER_BROKER_LISTENER_NAME: INTERNAL
      KAFKA_CONTROLLER_LISTENER_NAMES: CONTROLLER
      KAFKA_OFFSETS_TOPIC_REPLICATION_FACTOR: 1
      KAFKA_TRANSACTION_STATE_LOG_REPLICATION_FACTOR: 1
      KAFKA_TRANSACTION_STATE_LOG_MIN_ISR: 1
      KAFKA_GROUP_INITIAL_REBALANCE_DELAY_MS: 0
      KAFKA_AUTO_CREATE_TOPICS_ENABLE: "false"
    healthcheck:
      test: ["CMD-SHELL", "/opt/kafka/bin/kafka-broker-api-versions.sh --bootstrap-server localhost:19092 > /dev/null 2>&1"]
      interval: 5s
      timeout: 10s
      retries: 20
```

Message schemas

Request (every request carries `correlation_id`, `reply_to`, `operation` and `data`) – real message from `homework_demo.py`:

```
{
  "correlation_id": "05a2be9d-a396-45c6-a5d5-150f0120df1b",
  "reply_to": "s9871_entity_responses",
  "operation": "CREATE_ENTITY",
  "data": {
    "fixture_name": "Milpitas Lions vs San Jose Tigers",
    "teams_players": "Milpitas Lions (A. Singh, R. Gomez); San Jose Tigers (L. Chen, M. Okafor)",
    "submitter_email": "league-office@leaguemail.com",
    "description": "Rivalry match, gates open 9am, toss at 9:30am.",
    "category": "Soccer",
    "agree_terms": true
  }
}
```

Success response to CREATE_ENTITY (echoes the correlation_id, adds the generated `entity_id`):

```
{
  "correlation_id": "05a2be9d-a396-45c6-a5d5-150f0120df1b",
  "success": true,
  "entity_id": "338bce0e-6c21-4cef-b569-34deab9da5cd",
  "message": "Entity created"
}
```

GET_ENTITY request and its success response:

```
{
  "correlation_id": "2298384e-4747-4ff5-8d6f-6e65f08402c2",
  "reply_to": "s9871_entity_responses",
  "operation": "GET_ENTITY",
  "data": {
    "entity_id": "338bce0e-6c21-4cef-b569-34deab9da5cd"
  }
}
{
  "correlation_id": "2298384e-4747-4ff5-8d6f-6e65f08402c2",
  "success": true,
  "entity": {
    "entity_id": "338bce0e-6c21-4cef-b569-34deab9da5cd",
    "fixture_name": "Milpitas Lions vs San Jose Tigers",
    "teams_players": "Milpitas Lions (A. Singh, R. Gomez); San Jose Tigers (L. Chen, M. Okafor)",
    "submitter_email": "league-office@leaguemail.com",
    "description": "Rivalry match, gates open 9am, toss at 9:30am.",
    "category": "Soccer",
    "agree_terms": true
```

```
  }
}
```

Error responses (real output of API calls; every error carries the original correlation_id, and a caller-supplied `X-Correlation-ID` comes back unchanged, see `my-fixed-id-1`):

```
--- get unknown
{"correlation_id":"ba928f15-43c6-4457-afd0-765ae86d6a7d","success":false,"error":"Entity not found"}
--- unknown op
{"correlation_id":"cdff8db4-c433-4415-b34c-980ff94dbb5b","success":false,"error":"Unknown operation"}
--- empty name
{"correlation_id":"cf69b742-7e7e-4e15-99e0-00a235c83122","success":false,"error":"fixture_name is required and must be a non-empty string"}
--- extra field
{"correlation_id":"df70ffc7-1793-4c13-b2ec-0eca57778181","success":false,"error":"Unknown field(s): hax"}
--- agree false
{"correlation_id":"5c0e17e5-15ef-4fc5-b35a-b2152e544487","success":false,"error":"agree_terms must be true"}
--- caller corr id
{"correlation_id":"my-fixed-id-1","success":false,"error":"Unknown operation"}
```

Complete correlation_id flow (CREATE_ENTITY in the demo log)

Hop	Where	correlation_id
1	client creates and sends	`05a2be9d-a396-45c6-a5d5-150f0120df1b`
2	worker receives	`05a2be9d-a396-45c6-a5d5-150f0120df1b`
3	worker processes	`05a2be9d-a396-45c6-a5d5-150f0120df1b`
4	worker replies	`05a2be9d-a396-45c6-a5d5-150f0120df1b`
5	client receives the response	`05a2be9d-a396-45c6-a5d5-150f0120df1b`

The same 36-character value appears at all five hops (checked by the script that builds this report); the response is matched to the waiting caller only by this value.

Validation, processing and reply code

```
def validate_fixture(data) -> tuple[dict | None, str | None]:
    """Return (clean_fixture, None) or (None, error). Rejects missing, empty, wrong-type and unknown fields."""
    if not isinstance(data, dict):
        return None, "data must be an object"
    unknown = sorted(set(data) - set(FIXTURE_FIELDS))
    if unknown:
        return None, f"Unknown field(s): {', '.join(unknown)}"
    clean: dict = {}
    for field in FIXTURE_FIELDS[:4]:
        value = data.get(field)
        if not isinstance(value, str) or not value.strip():
            return None, f"{field} is required and must be a non-empty string"
        clean[field] = value.strip()
    if len(clean["fixture_name"]) > 255 or len(clean["teams_players"]) > 255:
        return None, "fixture_name and teams_players must be at most 255 characters"
    if not _EMAIL.match(clean["submitter_email"]):
        return None, "submitter_email is not a valid email address"
    category = data.get("category")
    if not isinstance(category, str) or category.strip().lower() not in CATEGORIES:
        return None, f"category must be one of: {', '.join(CATEGORIES.values())}"
    clean["category"] = CATEGORIES[category.strip().lower()]
    if data.get("agree_terms") is not True:
        return None, "agree_terms must be true"
    clean["agree_terms"] = True
    return clean, None

def handle_request(request: dict) -> dict:
    """Pure request -> response logic (no Kafka). The caller adds nothing: correlation_id is set here."""
    response: dict = {"correlation_id": request.get("correlation_id")}
    operation, data = request.get("operation"), request.get("data")
    if operation == "CREATE_ENTITY":
        fixture, error = validate_fixture(data)
        if error:
            return {**response, "success": False, "error": error}
        entity_id = str(uuid.uuid4())
        store[entity_id] = {"entity_id": entity_id, **fixture}
        return {**response, "success": True, "entity_id": entity_id, "message": "Entity created"}
    if operation == "GET_ENTITY":
        entity_id = data.get("entity_id") if isinstance(data, dict) else None
        if not isinstance(entity_id, str) or not entity_id:
            return {**response, "success": False, "error": "entity_id is required"}
        if entity_id not in store:
            return {**response, "success": False, "error": "Entity not found"}
        return {**response, "success": True, "entity": store[entity_id]}
    return {**response, "success": False, "error": "Unknown operation"}
```

```
        response = handle_request(request)
        log.info("PROCESSED corr=%s op=%s success=%s %s", corr, request.get("operation"), response["success"],
                 response.get("error") or response.get("message") or "entity returned")
        target = reply_topic(request)
        delivered = []
        producer.produce(target, key=corr, value=json.dumps(response),
                         on_delivery=lambda err, m: delivered.append((err, m)))
        producer.flush(10)
        err, sent = delivered[0] if delivered else ("not delivered", None)
        if err:
            log.error("REPLY FAILED corr=%s: %s (offset not committed, will be redelivered)", corr, err)
            continue
        log.info("REPLIED topic=%s partition=%d offset=%d corr=%s success=%s", sent.topic(), sent.partition(),
                 sent.offset(), corr, response["success"])
        consumer.commit(msg, asynchronous=False)
```

```
    def request(self, operation: str, data: dict, correlation_id: str | None = None, timeout: float = 10.0) -> dict:
        corr = correlation_id or str(uuid.uuid4())  # created once, never changed afterwards
        message = {"correlation_id": corr, "reply_to": RESPONSE_TOPIC, "operation": operation, "data": data}
        waiter = (threading.Event(), [])
        with self._lock:
            self._pending[corr] = waiter

        def delivered(err, msg):
            if err:
                self.log.error("SEND FAILED corr=%s: %s", corr, err)
            else:
                self.log.info("SENT request topic=%s partition=%d offset=%d corr=%s op=%s body=%s", msg.topic(),
                              msg.partition(), msg.offset(), corr, operation, msg.value().decode())

        self.producer.produce(REQUEST_TOPIC, key=corr, value=json.dumps(message), on_delivery=delivered)
        self.producer.flush(10)
        try:
            if not waiter[0].wait(timeout):
                return {"correlation_id": corr, "success": False, "error": "Timed out waiting for worker response"}
            return waiter[1][0]
        finally:
            with self._lock:
                self._pending.pop(corr, None)

async def call(operation: str, data, correlation_id: str | None) -> JSONResponse:
    log.info("HTTP -> Kafka op=%s", operation)
    response = await run_in_threadpool(rpc.request, operation, data, correlation_id)
    log.info("Kafka -> HTTP corr=%s success=%s", response["correlation_id"], response["success"])
    if response["success"]:
        status = 200
    elif response.get("error") == "Entity not found":
        status = 404
    elif response.get("error", "").startswith("Timed out"):
        status = 504
    else:
        status = 400
    return JSONResponse(response, status_code=status, headers={"X-Correlation-ID": response["correlation_id"]})
```

Required evidence

1. Docker containers running – `docker ps` shows the Kafka, worker and FastAPI containers; the worker log shows "Consumer is ready and listening..." and the FastAPI log shows the server started at http://127.0.0.1:8871.

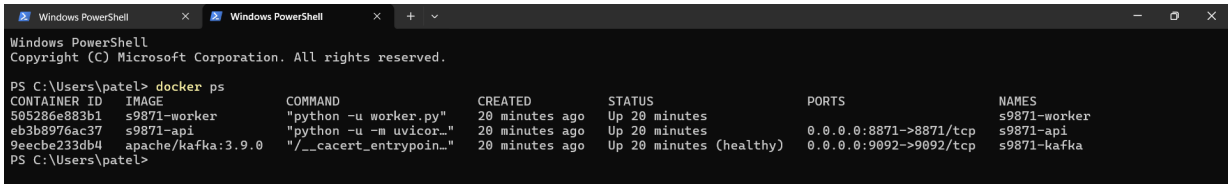


Figure 1: `docker ps`*: s9871-kafka (healthy),* `s9871-worker` *and* `s9871-api` *(port 8871) running.*

```
PS C:\Users\patel> docker logs s9871-worker
2026-10-29 11:10:59.845 [worker] Assigned partitions: [0]
2026-10-29 11:10:59.845 [worker] Consumer is ready and listening... topic=s9871_entity_requests group=s9871_entity_workers
2026-10-29 11:11:02.608 [worker] RECEIVED topic=s9871_entity_requests partition=0 offset=0 corr=05a2be9d-a396-45c6-a5d5-150f0120df1b op=CREATE_ENTITY
2026-10-29 11:11:02.608 [worker] PROCESSED corr=05a2be9d-a396-45c6-a5d5-150f0120df1b op=CREATE_ENTITY success=True Entity created
2026-10-29 11:11:02.612 [worker] REPLIED topic=s9871_entity_responses partition=0 offset=0 corr=05a2be9d-a396-45c6-a5d5-150f0120df1b success=True
2026-10-29 11:11:03.162 [worker] RECEIVED topic=s9871_entity_requests partition=0 offset=1 corr=2298384e-4747-4ff5-8d6f-6e65f08402c2 op=GET_ENTITY
2026-10-29 11:11:03.162 [worker] PROCESSED corr=2298384e-4747-4ff5-8d6f-6e65f08402c2 op=GET_ENTITY success=True entity returned
2026-10-29 11:11:03.164 [worker] REPLIED topic=s9871_entity_responses partition=0 offset=1 corr=2298384e-4747-4ff5-8d6f-6e65f08402c2 success=True
2026-10-29 11:11:05.216 [worker] RECEIVED topic=s9871_entity_requests partition=0 offset=2 corr=57c59c9c-3ea5-4e7f-90b7-d3b7620bde0b op=CREATE_ENTITY
2026-10-29 11:11:05.217 [worker] PROCESSED corr=57c59c9c-3ea5-4e7f-90b7-d3b7620bde0b op=CREATE_ENTITY success=True Entity created
2026-10-29 11:11:05.218 [worker] REPLIED topic=s9871_entity_responses partition=0 offset=2 corr=57c59c9c-3ea5-4e7f-90b7-d3b7620bde0b success=True
2026-10-29 11:11:05.418 [worker] RECEIVED topic=s9871_entity_requests partition=0 offset=3 corr=dbab4e8b-6f8a-44b6-9c1d-cbc2095ae2f3 op=GET_ENTITY
2026-10-29 11:11:05.418 [worker] PROCESSED corr=dbab4e8b-6f8a-44b6-9c1d-cbc2095ae2f3 op=GET_ENTITY success=True entity returned
2026-10-29 11:11:05.420 [worker] REPLIED topic=s9871_entity_responses partition=0 offset=3 corr=dbab4e8b-6f8a-44b6-9c1d-cbc2095ae2f3 success=True
2026-10-29 11:11:05.452 [worker] RECEIVED topic=s9871_entity_requests partition=0 offset=4 corr=ba928f15-43c6-4457-afd0-765ae86d6a7d op=GET_ENTITY
2026-10-29 11:11:05.452 [worker] PROCESSED corr=ba928f15-43c6-4457-afd0-765ae86d6a7d op=GET_ENTITY success=False Entity not found
2026-10-29 11:11:05.454 [worker] REPLIED topic=s9871_entity_responses partition=0 offset=4 corr=ba928f15-43c6-4457-afd0-765ae86d6a7d success=False
2026-10-29 11:11:05.485 [worker] RECEIVED topic=s9871_entity_requests partition=0 offset=5 corr=cdff8db4-c433-4415-b34c-980ff94dbb5b op=DELETE_ENTITY
2026-10-29 11:11:05.485 [worker] PROCESSED corr=cdff8db4-c433-4415-b34c-980ff94dbb5b op=DELETE_ENTITY success=False Unknown operation
2026-10-29 11:11:05.486 [worker] REPLIED topic=s9871_entity_responses partition=0 offset=5 corr=cdff8db4-c433-4415-b34c-980ff94dbb5b success=False
2026-10-29 11:11:05.507 [worker] RECEIVED topic=s9871_entity_requests partition=0 offset=6 corr=cf69b742-7e7e-4e15-99e0-00a235c83122 op=CREATE_ENTITY
2026-10-29 11:11:05.507 [worker] PROCESSED corr=cf69b742-7e7e-4e15-99e0-00a235c83122 op=CREATE_ENTITY success=False fixture_name is required and must be a n
on-empty string
2026-10-29 11:11:05.509 [worker] REPLIED topic=s9871_entity_responses partition=0 offset=6 corr=cf69b742-7e7e-4e15-99e0-00a235c83122 success=False
2026-10-29 11:11:05.528 [worker] RECEIVED topic=s9871_entity_requests partition=0 offset=7 corr=df70ffc7-1793-4c13-b2ec-0eca57778181 op=CREATE_ENTITY
2026-10-29 11:11:05.528 [worker] PROCESSED corr=df70ffc7-1793-4c13-b2ec-0eca57778181 op=CREATE_ENTITY success=False Unknown field(s): hax
2026-10-29 11:11:05.529 [worker] REPLIED topic=s9871_entity_responses partition=0 offset=7 corr=df70ffc7-1793-4c13-b2ec-0eca57778181 success=False
2026-10-29 11:11:05.549 [worker] RECEIVED topic=s9871_entity_requests partition=0 offset=8 corr=5c0e17e5-15ef-4fc5-b35a-b2152e544487 op=CREATE_ENTITY
2026-10-29 11:11:05.549 [worker] PROCESSED corr=5c0e17e5-15ef-4fc5-b35a-b2152e544487 op=CREATE_ENTITY success=False agree_terms must be true
2026-10-29 11:11:05.551 [worker] REPLIED topic=s9871_entity_responses partition=0 offset=8 corr=5c0e17e5-15ef-4fc5-b35a-b2152e544487 success=False
2026-10-29 11:11:05.583 [worker] RECEIVED topic=s9871_entity_requests partition=0 offset=9 corr=my-fixed-id-1 op=NOPE
2026-10-29 11:11:05.583 [worker] PROCESSED corr=my-fixed-id-1 op=NOPE success=False Unknown operation
2026-10-29 11:11:05.584 [worker] REPLIED topic=s9871_entity_responses partition=0 offset=9 corr=my-fixed-id-1 success=False
PS C:\Users\patel>
```

Figure 2: `docker logs s9871-worker`*: "Consumer is ready and listening...", then RECEIVED / PROCESSED / REPLIED for every request.*

```
PS C:\Users\patel> docker logs s9871-api
INFO:     Started server process [1]
INFO:     Waiting for application startup.
2026-10-29 11:10:57.244 [api] Kafka client ready. FastAPI server started at http://127.0.0.1:8871
INFO:     Application startup complete.
INFO:     Uvicorn running on http://0.0.0.0:8871 (Press CTRL+C to quit)
2026-10-29 11:11:05.211 [api] HTTP -> Kafka op=CREATE_ENTITY
2026-10-29 11:11:05.216 [api] SENT request topic=s9871_entity_requests partition=0 offset=2 corr=57c59c9c-3ea5-4e7f-90b7-d3b7620bde0b op=CREATE_ENTITY body=
{"correlation_id": "57c59c9c-3ea5-4e7f-90b7-d3b7620bde0b", "reply_to": "s9871_entity_responses", "operation": "CREATE_ENTITY", "data": {"fixture_name": "Lio
ns vs Tigers", "teams_players": "Lions; Tigers", "submitter_email": "a@b.com", "description": "d", "category": "volleyball", "agree_terms": true}}
2026-10-29 11:11:05.219 [api] RECEIVED response topic=s9871_entity_responses partition=0 offset=2 corr=57c59c9c-3ea5-4e7f-90b7-d3b7620bde0b success=True bod
y={"correlation_id": "57c59c9c-3ea5-4e7f-90b7-d3b7620bde0b", "success": true, "entity_id": "fc1a5851-b853-4cf7-bbc6-520c95c95a48", "message": "Entity create
d"}
2026-10-29 11:11:05.219 [api] Kafka -> HTTP corr=57c59c9c-3ea5-4e7f-90b7-d3b7620bde0b success=True
INFO:     172.21.0.1:35470 - "POST /fixtures HTTP/1.1" 200 OK
2026-10-29 11:11:05.414 [api] HTTP -> Kafka op=GET_ENTITY
2026-10-29 11:11:05.417 [api] SENT request topic=s9871_entity_requests partition=0 offset=3 corr=dbab4e8b-6f8a-44b6-9c1d-cbc2095ae2f3 op=GET_ENTITY body={"c
orrelation_id": "dbab4e8b-6f8a-44b6-9c1d-cbc2095ae2f3", "reply_to": "s9871_entity_responses", "operation": "GET_ENTITY", "data": {"entity_id": "fc1a5851-b85
3-4cf7-bbc6-520c95c95a48"}}
```

Figure 3: `docker logs s9871-api`*: "FastAPI server started at http://127.0.0.1:8871" and the HTTP -> Kafka -> HTTP flow of a CREATE_ENTITY call.*

2. CREATE_ENTITY and GET_ENTITY execution and **3. message-flow logs** – `homework_demo.py` (the Kafka client): the request sent to `s9871_entity_requests`, the worker's receive / process / reply lines, and the response received by the client, for both operations. (These two screenshots are a later run of the same script, so the offsets are 10 and 11; the text excerpt below is the first run.)

```
PS C:\Users\patel> cd .\data260-9871\
PS C:\Users\patel\data260-9871> cd .\code\kafka_hw8\
PS C:\Users\patel\data260-9871\code\kafka_hw8> ..\venv\Scripts\python.exe .\homework_demo.py
Lines are printed in causal order (send -> worker -> client receive). The worker's clock runs in the
Docker VM and can differ from this host's clock by a few hundred ms, so compare offsets, not timestamps.

=== Step 1: create one correlation_id ===
correlation_id = 582de942-bcc3-4e4b-bf81-9321fbedf037

=== Steps 2-6: CREATE_ENTITY (correlation_id=582de942-bcc3-4e4b-bf81-9321fbedf037) ===
2026-10-29 11:35:10.277 [client] Sending CREATE_ENTITY to s9871_entity_requests
2026-10-29 11:35:10.369 [client] SENT request topic=s9871_entity_requests partition=0 offset=10 corr=582de942-bcc3-4e4b-bf81-9321fbedf037 op=CREATE_ENTITY b
ody={"correlation_id": "582de942-bcc3-4e4b-bf81-9321fbedf037", "reply_to": "s9871_entity_responses", "operation": "CREATE_ENTITY", "data": {"fixture_name":
"Milpitas Lions vs San Jose Tigers", "teams_players": "Milpitas Lions (A. Singh, R. Gomez); San Jose Tigers (L. Chen, M. Okafor)", "submitter_email": "leagu
e-office@leaguemail.com", "description": "Rivalry match, gates open 9am, toss at 9:30am.", "category": "Soccer", "agree_terms": true}}
  [from docker logs s9871-worker] 2026-10-29 11:35:10.468 [worker] RECEIVED topic=s9871_entity_requests partition=0 offset=10 corr=582de942-bcc3-4e4b-bf81-9
321fbedf037 op=CREATE_ENTITY
  [from docker logs s9871-worker] 2026-10-29 11:35:10.470 [worker] PROCESSED corr=582de942-bcc3-4e4b-bf81-9321fbedf037 op=CREATE_ENTITY success=True Entity
created
  [from docker logs s9871-worker] 2026-10-29 11:35:10.472 [worker] REPLIED topic=s9871_entity_responses partition=0 offset=10 corr=582de942-bcc3-4e4b-bf81-9
321fbedf037 success=True
2026-10-29 11:35:10.376 [client] RECEIVED response topic=s9871_entity_responses partition=0 offset=10 corr=582de942-bcc3-4e4b-bf81-9321fbedf037 success=True
 body={"correlation_id": "582de942-bcc3-4e4b-bf81-9321fbedf037", "success": true, "entity_id": "c0f0f8a3-039b-47a4-9344-cdfdbd8434e3", "message": "Entity cr
eated"}
2026-10-29 11:35:10.376 [client] Response returned to caller: {"correlation_id": "582de942-bcc3-4e4b-bf81-9321fbedf037", "success": true, "entity_id": "c0f0
f8a3-039b-47a4-9344-cdfdbd8434e3", "message": "Entity created"}
```

Figure 4: `homework_demo.py`*, steps 1-6: one correlation_id, CREATE_ENTITY sent, worker receives / processes / replies, client receives the response with the generated entity_id.*



Figure 5: `homework_demo.py`, steps 7-10: entity_id extracted, GET_ENTITY full sequence, the retrieved entity and the correlation_id checks.

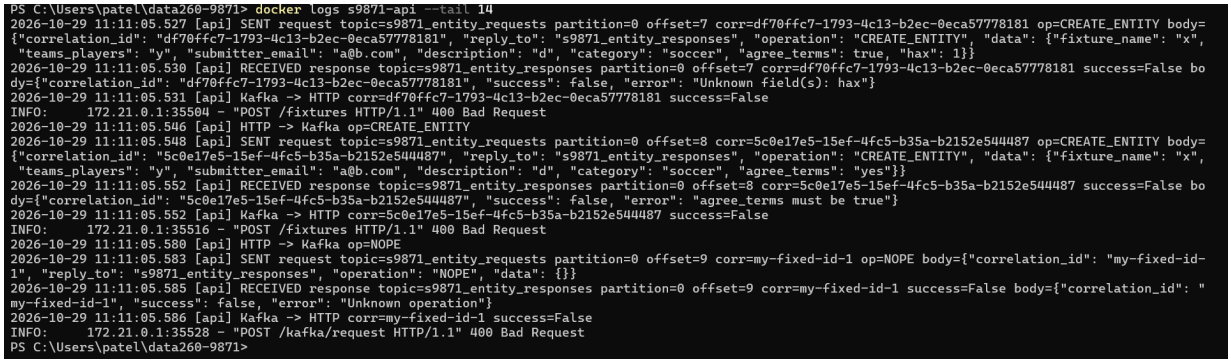


Figure 6: FastAPI wrapper (`docker logs s9871-api`): error requests (unknown field, `agree_terms`, unknown operation) go HTTP -> Kafka -> HTTP with the correlation_id unchanged.

4. Full sequence – log excerpt, CREATE_ENTITY then GET_ENTITY, in order (send, receive, process, reply, final client response; request/response bodies shortened; the worker runs in the Docker VM, so its clock differs from the client's – here the worker's lines are stamped about 0.3 s earlier than the client's send, in other runs a few tens of ms later – compare the offsets, not the timestamps):

```
2026-10-29 11:11:02.857 [client] Sending CREATE_ENTITY to s9871_entity_requests
2026-10-29 11:11:02.895 [client] SENT request topic=s9871_entity_requests partition=0 offset=0 corr=05a2be9d-a396-45c6-a5d5-150f0120df1b op=
      CREATE_ENTITY body={...}
  2026-10-29 11:11:02.608 [worker] RECEIVED topic=s9871_entity_requests partition=0 offset=0 corr=05a2be9d-a396-45c6-a5d5-150f0120df1b op=CREATE_ENTITY
  2026-10-29 11:11:02.608 [worker] PROCESSED corr=05a2be9d-a396-45c6-a5d5-150f0120df1b op=CREATE_ENTITY success=True Entity created
  2026-10-29 11:11:02.612 [worker] REPLIED topic=s9871_entity_responses partition=0 offset=0 corr=05a2be9d-a396-45c6-a5d5-150f0120df1b success=True
2026-10-29 11:11:02.909 [client] RECEIVED response topic=s9871_entity_responses partition=0 offset=0 corr=05a2be9d-a396-45c6-a5d5-150f0120df1b success=
      True body={...}
2026-10-29 11:11:02.909 [client] Response returned to caller: {"correlation_id": "05a2be9d-a396-45c6-a5d5-150f0120df1b", "success": true, "entity_id":
      ...
2026-10-29 11:11:03.469 [client] Sending GET_ENTITY to s9871_entity_requests
2026-10-29 11:11:03.472 [client] SENT request topic=s9871_entity_requests partition=0 offset=1 corr=2298384e-4747-4ff5-8d6f-6e65f08402c2 op=GET_ENTITY
      body={...}
  2026-10-29 11:11:03.162 [worker] RECEIVED topic=s9871_entity_requests partition=0 offset=1 corr=2298384e-4747-4ff5-8d6f-6e65f08402c2 op=GET_ENTITY
  2026-10-29 11:11:03.162 [worker] PROCESSED corr=2298384e-4747-4ff5-8d6f-6e65f08402c2 op=GET_ENTITY success=True entity returned
  2026-10-29 11:11:03.164 [worker] REPLIED topic=s9871_entity_responses partition=0 offset=1 corr=2298384e-4747-4ff5-8d6f-6e65f08402c2 success=True
2026-10-29 11:11:03.476 [client] RECEIVED response topic=s9871_entity_responses partition=0 offset=1 corr=2298384e-4747-4ff5-8d6f-6e65f08402c2 success=
      True body={...}
2026-10-29 11:11:03.476 [client] Response returned to caller: {"correlation_id": "2298384e-4747-4ff5-8d6f-6e65f08402c2", "success": true, "entity": {"
      ...
```

Part 2 – Three-Agent Kafka Pipeline

Planner, Writer and Reviewer run as separate processes and never call each other: each reads one topic and writes the next (`s9871_inbox` -> `s9871_tasks` -> `s9871_drafts` -> `s9871_final`). The model is qwen3:8b through the HW1 `ModelClient` (thinking off). All three share one consume -> handle -> produce -> commit loop (`run_stage`) and copy the correlation_id unchanged. The four commands of the assignment (`python planner.py`, `writer.py`, `reviewer.py`, `send_question.py`) were run as four separate operating-system processes; each agent's output is saved in its own file, `raw/part2_planner.log`, `part2_writer.log`, `part2_reviewer.log` and `part2_send_question.txt`.

Topic creation (Kafka CLI)

```
$ kafka-topics.sh --create --topic s9871_inbox --partitions 1 --replication-factor 1 --bootstrap-server localhost:9092
Created topic s9871_inbox.
$ kafka-topics.sh --create --topic s9871_tasks --partitions 2 --replication-factor 1 --bootstrap-server localhost:9092
Created topic s9871_tasks.
$ kafka-topics.sh --create --topic s9871_drafts --partitions 1 --replication-factor 1 --bootstrap-server localhost:9092
Created topic s9871_drafts.
$ kafka-topics.sh --create --topic s9871_final --partitions 1 --replication-factor 1 --bootstrap-server localhost:9092
Created topic s9871_final.
$ kafka-topics.sh --list --bootstrap-server localhost:9092
Topic: s9871_inbox    PartitionCount: 1    ReplicationFactor: 1    Configs:
Topic: s9871_tasks    PartitionCount: 2    ReplicationFactor: 1    Configs:
Topic: s9871_drafts    PartitionCount: 1    ReplicationFactor: 1    Configs:
Topic: s9871_final    PartitionCount: 1    ReplicationFactor: 1    Configs:
```

Planner, Writer and Reviewer code

```
SYSTEM = (
    "You are the Planner for a community sports league office. Break the user's question into exactly 3 short "
    "steps that a writer will follow to answer it. Reply with JSON only, in this shape: "
    '{"steps": ["<step one>", "<step two>", "<step three>"]}'
)

def plan(message: dict) -> dict:
    for _ in range(2):  # one retry if the model's reply is not the requested JSON
        try:
            steps = parse_json_object(ask_model(SYSTEM, message["question"]))["steps"]
            if isinstance(steps, list) and len(steps) == 3 and all(isinstance(s, str) and s.strip() for s in steps):
                return {"question": message["question"], "plan": steps}
        except (ValueError, KeyError):
            pass
    raise RuntimeError("planner could not get a valid 3-step plan from the model")
```

```
SYSTEM = (
    "You are the Writer for a community sports league office. Answer the question by following the plan "
    "step by step. Write plain prose, at most 120 words, no markdown headings."
)

def write(message: dict) -> dict:
    plan = message.get("plan")
    if not isinstance(plan, list) or not plan or not all(isinstance(s, str) and s.strip() for s in plan):
        raise ValueError("plan must be a non-empty list of non-empty strings")
    corr = message["correlation_id"]
    log.info("WORK start corr=%s", corr)
    if STUB:
        time.sleep(float(message.get("work_s", 0)))
        draft = "[stub draft] " + " ".join(plan)
    else:
        steps = "\n".join(f"{i}. {s}" for i, s in enumerate(plan, 1))
        draft = ask_model(SYSTEM, f"Question: {message['question']}\n\nPlan:\n{steps}", temperature=0.5)
    log.info("WORK done corr=%s", corr)
    out = {"question": message["question"], "plan": plan, "draft": draft}
    out.update({k: message[k] for k in ("entity_id", "entity_seq") if k in message})
    return out
```

```
SYSTEM = (
    "You are the Reviewer for a community sports league office. Check that the draft answers the question, "
    "follows the plan and is clear. Reply with JSON only: "
    '{"approved": true or false, "comment": "<one short sentence>"}'
)

def review(message: dict) -> dict:
    draft = message["draft"].strip()
    if not draft:  # deterministic guard: never approve an empty answer, whatever the model says
        return {"status": "rejected", "answer": draft, "review": "draft is empty"}
    try:
        verdict = parse_json_object(ask_model(
            SYSTEM, f"Question: {message['question']}\n\nPlan: {message['plan']}\n\nDraft:\n{draft}"))
        approved, comment = verdict.get("approved") is True, str(verdict.get("comment", ""))
    except ValueError:
        approved, comment = False, "reviewer could not parse the model's verdict"
    return {"status": "approved" if approved else "rejected", "answer": draft, "review": comment}
```

The shared loop that reads the input topic, calls the handler, produces to the next topic and commits (the handler is the function above; `send_question.py` is the sender):

```
def run_stage(stage: str, in_topic: str, out_topic: str, group: str, handler) -> None:
    """handler(message: dict) -> dict. correlation_id is carried over from the input, never regenerated."""
    name, group = stage_name(stage), group + GROUP_SUFFIX
    log = setup_logging(name)
    store = ProcessedStore(stage) if IDEMPOTENT else None
    consumer = Consumer({"bootstrap.servers": BOOTSTRAP, "group.id": group, "auto.offset.reset": "earliest",
                         "enable.auto.commit": False, "session.timeout.ms": 10000})
    producer = Producer({"bootstrap.servers": BOOTSTRAP, "acks": "all"})
    running = [True]
    signal.signal(signal.SIGINT, lambda *_: running.__setitem__(0, False))
    signal.signal(signal.SIGTERM, lambda *_: running.__setitem__(0, False))
```

One complete message journey, inbox to final

Each stage's real log lines for one question, in the order they happened (one host, so one clock; message bodies shortened):

```
2026-10-29 11:55:56.583 [you] SENT topic=s9871_inbox partition=0 offset=0 corr=494d5989-7211-4b44-af22-7a551d84d896 body={...}
2026-10-29 11:55:56.583 [planner] RECEIVED topic=s9871_inbox partition=0 offset=0 corr=494d5989-7211-4b44-af22-7a551d84d896
2026-10-29 11:56:08.974 [planner] PROCESSED corr=494d5989-7211-4b44-af22-7a551d84d896
2026-10-29 11:56:09.022 [planner] SENT topic=s9871_tasks partition=0 offset=0 corr=494d5989-7211-4b44-af22-7a551d84d896 body={...}
2026-10-29 11:56:09.023 [writer] RECEIVED topic=s9871_tasks partition=0 offset=0 corr=494d5989-7211-4b44-af22-7a551d84d896
2026-10-29 11:56:13.545 [writer] PROCESSED corr=494d5989-7211-4b44-af22-7a551d84d896
2026-10-29 11:56:13.595 [writer] SENT topic=s9871_drafts partition=0 offset=0 corr=494d5989-7211-4b44-af22-7a551d84d896 body={...}
2026-10-29 11:56:13.595 [reviewer] RECEIVED topic=s9871_drafts partition=0 offset=0 corr=494d5989-7211-4b44-af22-7a551d84d896
2026-10-29 11:56:16.423 [reviewer] PROCESSED corr=494d5989-7211-4b44-af22-7a551d84d896
2026-10-29 11:56:16.429 [reviewer] SENT topic=s9871_final partition=0 offset=0 corr=494d5989-7211-4b44-af22-7a551d84d896 body={...}
2026-10-29 11:56:16.429 [you] RECEIVED topic=s9871_final partition=0 offset=0 corr=494d5989-7211-4b44-af22-7a551d84d896
```

Final message from s9871_final

```
{"correlation_id": "494d5989-7211-4b44-af22-7a551d84d896", "question": "What should a league office check before
    scheduling a Soccer fixture on a shared field?"}
2026-10-29 11:56:16.429 [you] RECEIVED topic=s9871_final partition=0 offset=0 corr=494d5989-7211-4b44-af22-7
    a551d84d896
{
  "status": "approved",
  "answer": "Before scheduling a soccer fixture on a shared field, the league office should check the field's
    availability and booking status for the scheduled time. They should also verify that the field meets safety
    and maintenance standards to ensure a safe playing environment. Lastly, they must confirm there are no
    conflicting events or reservations for the field on that date to avoid scheduling overlaps.",
  "review": "Draft follows the plan and is clear.",
  "correlation_id": "494d5989-7211-4b44-af22-7a551d84d896"
}
```

How Kafka helped the agents coordinate

Planner, Writer and Reviewer are three separate processes that never call each other; each only reads one topic and writes the next one (`s9871_inbox` -> `s9871_tasks` -> `s9871_drafts` -> `s9871_final`). One real question took 19.8 s end to end with qwen3:8b (Planner 12.4 s, Writer 4.6 s, Reviewer 2.8 s), and the same correlation_id is on every message from the question to the final answer.

- **Decoupling.** An agent only needs the topic names, not the address or API of the next agent. A stage that is down does not lose work: its input waits in the topic and is read when it comes back (shown for the Writer in Part 3).
- **Scaling.** `s9871_tasks` has two partitions, so two Writers in one consumer group split the work with no change to the Writer's code (Part 3, ordering).
- **Durable hand-offs and offsets.** Every hand-off is a persisted, replayable message, and each agent commits its input offset only after its output is delivered, so a crash causes a re-read, not a lost message.

- **Correlation.** The correlation_id lets the final answer be matched to the question without any shared state between the agents.

Part 3 – Failure Behavior

Every "before fix" below is a real run of the same code with the fix switched off. `HW8_MODE=naive|fixed` (default `fixed`) sets three behaviours of the shared loop; each can also be overridden alone. The poison-message "before" run keeps commit-after, so that only the retry policy differs.

Behaviour	Fixed	Naive (before fix)
Idempotency	skip a correlation_id already handled (SQLite store)	handle every delivery
Offset commit	after the output message is delivered	before processing (at-most-once)
Poison message	3 attempts, then `s9871_dead_letter`	retry forever, never commit

Disclosure: model-free Writer mode. To make the failure tests fast and repeatable, the Writer ran with `WRITER_MODE=stub` in every Part 3 test: it skips the qwen3:8b call and sleeps for the message's own `work_s` seconds instead, writing a fixed draft. Validation, Kafka reads and writes, retries, commits and the idempotency store are the same code as in the real Writer. The Part 2 journey above used the real model, and `verify_hw08.py` runs one real-model journey too. The Writers in these tests are real processes (the kill test is a hard kill with `taskkill /F`), started by `failure_tests.py`, which resets the topics before each run.

Failure table (before / after fix)

Failure scenario	Before fix (naive)	After fix
Duplicate delivery	one correlation_id replayed -> 2 deliveries, 2 work executions, **2 drafts** in `s9871_drafts`	same replay -> 2 deliveries, 1 work execution, 1 `DUPLICATE ... skipped`, **1 draft**
Consumer killed mid-work	offset committed *before* processing: after the kill committed offset = 1, lag 0; after restart redelivered = False, drafts = **0** (message silently lost)	offset committed *after* success: after the kill committed offset = none; after restart redelivered = True, drafts = **1**, then committed offset = 1 (lag 0)
Poison message	retried **29+ times without limit**; good message behind it processed = False; consumer-group lag = 2	**3 attempts**, then 1 message in `s9871_dead_letter` (attempts field = 3); good message processed = True; lag 0
Two partitions / two consumers	no key: 3 of 4 entities processed out of order (12 inversions); 3 entities split across both partitions	key = entity_id: 0 entities out of order (0 inversions); 0 entities split across partitions

Idempotency, offset-commit, retry/dead-letter and partition-key code

Settings and the durable processed-correlation_id store:

```
DEAD_LETTER = f"{PREFIX}_dead_letter"
MODE = os.environ.get("HW8_MODE", "fixed")
IDEMPOTENT = os.environ.get("HW8_IDEMPOTENT", "1" if MODE == "fixed" else "0") == "1"
COMMIT = os.environ.get("HW8_COMMIT", "after" if MODE == "fixed" else "before")
POISON = os.environ.get("HW8_POISON", "dlq" if MODE == "fixed" else "retry-forever")
MAX_ATTEMPTS = int(os.environ.get("HW8_MAX_ATTEMPTS", "3"))
RETRY_PAUSE_S = float(os.environ.get("HW8_RETRY_PAUSE_S", "0.5"))
GROUP_SUFFIX = os.environ.get("HW8_GROUP_SUFFIX", "")  # lets a test use a fresh consumer group
INSTANCE = os.environ.get("HW8_INSTANCE", "")  # "A", "B": tells two Writers apart in the logs
STATE_DIR = Path(os.environ.get("HW8_STATE_DIR", Path(__file__).resolve().parent / "state"))


class ProcessedStore:
    """Durable set of correlation_ids a stage has finished (SQLite file, shared by that stage's instances)."""

    def __init__(self, stage: str):
        STATE_DIR.mkdir(parents=True, exist_ok=True)
        self.db = sqlite3.connect(STATE_DIR / f"processed_{stage}.db", timeout=30, isolation_level=None)
        self.db.execute("CREATE TABLE IF NOT EXISTS processed (correlation_id TEXT PRIMARY KEY)")

    def seen(self, corr: str) -> bool:
        return self.db.execute("SELECT 1 FROM processed WHERE correlation_id=?", (corr,)).fetchone() is not None
```

```
    def mark(self, corr: str) -> None:
        self.db.execute("INSERT OR IGNORE INTO processed VALUES (?)", (corr,))
```

The loop body: commit-before (naive), the duplicate check, bounded retries and the dead-letter message, then produce, record the correlation_id and commit-after (fixed):

```
        if COMMIT == "before":
            commit(msg, "BEFORE processing")
        if store and store.seen(corr):
            log.info("DUPLICATE corr=%s already processed, skipped (no output, no side effect)", corr)
            if COMMIT == "after":
                commit(msg, "duplicate")
            continue

        attempt, out, error = 0, None, None
        while running[0]:
            attempt += 1
            try:
                out = handler(message)
                break
            except Exception as exc:  # any handler failure counts as a failed attempt
                error = f"{type(exc).__name__}: {exc}"
                limit = f"{attempt}/{MAX_ATTEMPTS}" if POISON == "dlq" else f"{attempt}/unbounded"
                log.warning("ATTEMPT %s failed corr=%s: %s", limit, corr, error)
                if POISON == "dlq" and attempt >= MAX_ATTEMPTS:
                    break
                time.sleep(RETRY_PAUSE_S)
        if out is None and not (POISON == "dlq" and attempt >= MAX_ATTEMPTS and error):
            break  # shutting down in the middle of retries: leave the offset uncommitted
        if out is None:
            letter = {"correlation_id": corr, "stage": stage, "error": error, "attempts": attempt,
                      "source_topic": msg.topic(), "source_partition": msg.partition(),
                      "source_offset": msg.offset(), "original": msg.value().decode("utf-8", "replace")}
            sent = send(DEAD_LETTER, corr, letter)
            if sent is None:
                continue
            log.info("DEAD-LETTERED corr=%s after %d attempts -> topic=%s partition=%d offset=%d error=%s", corr,
                     attempt, sent.topic(), sent.partition(), sent.offset(), error)
            if COMMIT == "after":
                commit(msg, "after dead-lettering")
            continue

        ...
        out["correlation_id"] = corr
        log.info("PROCESSED corr=%s %s%s", corr, where, tags(message))
        sent = send(out_topic, corr, out)
        if sent is None:
            continue
        log.info("SENT topic=%s partition=%d offset=%d corr=%s body=%s", sent.topic(), sent.partition(),
                 sent.offset(), corr, json.dumps(out))
        if store:
            store.mark(corr)
        if COMMIT == "after":
            commit(msg, "AFTER successful processing")
```

Partition key (the keyed ordering run sends each message with `key = entity_id`; the unkeyed run sends `key = None`):

```
    keyed = mode == "keyed"
    say(f"sending 20 messages, key={'entity_id' if keyed else 'none (null key)'}")
    landed = produce([(m["entity_id"] if keyed else None, m, None) for m in messages])

def produce(messages: list[tuple], flush: bool = True) -> list[dict]:
    """messages: (key, payload_dict_or_bytes, partition_or_None). Returns where each one landed."""
    producer = Producer({"bootstrap.servers": BOOTSTRAP, "acks": "all"})
    landed = []

    def done(err, msg):
        landed.append({"partition": msg.partition(), "offset": msg.offset(), "key": (msg.key() or b"").decode()})

    for key, payload, partition in messages:
        value = payload if isinstance(payload, bytes) else json.dumps(payload).encode()
        kwargs = {"partition": partition} if partition is not None else {}
        producer.produce(TASKS, key=key, value=value, on_delivery=done, **kwargs)
    producer.flush(15)
    return landed
```

I. Duplicate delivery – logs

The same message (same correlation_id, same bytes) is delivered a second time. Before: the Writer does the work twice and emits two drafts. After: the second delivery is logged as a duplicate, with no output and no side effect.

Before (naive):

```
2026-10-29 13:05:28.614 [writer-A] Consumer is ready and listening... topic=s9871_tasks group=s9871_writers_duplicate_naive_a7565d mode=naive
      idempotent=False commit=before poison=retry-forever
2026-10-29 13:05:29.226 [writer-A] RECEIVED topic=s9871_tasks partition=0 offset=0 corr=dup-naive-1d2538
2026-10-29 13:05:29.229 [writer-A] COMMITTED BEFORE processing offset=0 (next to read = 1)
2026-10-29 13:05:29.230 [writer-A] WORK start corr=dup-naive-1d2538
2026-10-29 13:05:29.234 [writer-A] SENT topic=s9871_drafts partition=0 offset=0 corr=dup-naive-1d2538 body={...}
2026-10-29 13:05:29.377 [writer-A] RECEIVED topic=s9871_tasks partition=0 offset=1 corr=dup-naive-1d2538
2026-10-29 13:05:29.378 [writer-A] COMMITTED BEFORE processing offset=1 (next to read = 2)
2026-10-29 13:05:29.378 [writer-A] WORK start corr=dup-naive-1d2538
2026-10-29 13:05:29.380 [writer-A] SENT topic=s9871_drafts partition=0 offset=1 corr=dup-naive-1d2538 body={...}
```

After (fixed):

```
2026-10-29 13:05:42.814 [writer-A] Consumer is ready and listening... topic=s9871_tasks group=s9871_writers_duplicate_fixed_159ace mode=fixed
      idempotent=True commit=after poison=dlq
2026-10-29 13:05:43.425 [writer-A] RECEIVED topic=s9871_tasks partition=0 offset=0 corr=dup-fixed-d22441
2026-10-29 13:05:43.426 [writer-A] WORK start corr=dup-fixed-d22441
2026-10-29 13:05:43.472 [writer-A] SENT topic=s9871_drafts partition=0 offset=0 corr=dup-fixed-d22441 body={...}
2026-10-29 13:05:43.481 [writer-A] COMMITTED AFTER successful processing offset=0 (next to read = 1)
2026-10-29 13:05:43.676 [writer-A] RECEIVED topic=s9871_tasks partition=0 offset=1 corr=dup-fixed-d22441
2026-10-29 13:05:43.676 [writer-A] DUPLICATE corr=dup-fixed-d22441 already processed, skipped (no output, no side effect)
2026-10-29 13:05:43.678 [writer-A] COMMITTED duplicate offset=1 (next to read = 2)
```

II. Consumer death – logs (Writer hard-killed 1.5 s into an 8 s job, then restarted)

Before (naive, offset committed before processing): the broker already counts the message as done, so after the restart nothing arrives and no draft is ever produced.

```
2026-10-29 19:01:43.024 [harness] started Writer A pid=64036 env={'HW8_MODE': 'naive'}
2026-10-29 19:01:45.954 [harness] HARD-KILLED Writer A pid=64036
2026-10-29 19:01:46.107 [harness] right after the kill: committed offsets={0: 1, 1: None} drafts_for_corr=0
2026-10-29 19:01:46.112 [harness] started Writer A pid=63972 env={'HW8_MODE': 'naive'}
--- writer 1 (killed) ---
2026-10-29 19:01:43.458 [writer-A] Consumer is ready and listening... topic=s9871_tasks group=s9871_writers_kill_naive_b91704 mode=naive idempotent=
      False commit=before poison=retry-forever
2026-10-29 19:01:44.077 [writer-A] RECEIVED topic=s9871_tasks partition=0 offset=0 corr=kill-naive-d5b1ef
2026-10-29 19:01:44.099 [writer-A] COMMITTED BEFORE processing offset=0 (next to read = 1)
2026-10-29 19:01:44.100 [writer-A] WORK start corr=kill-naive-d5b1ef
--- writer 2 (restarted; nothing received) ---
2026-10-29 19:01:53.973 [writer-A] Consumer is ready and listening... topic=s9871_tasks group=s9871_writers_kill_naive_b91704 mode=naive idempotent=
      False commit=before poison=retry-forever
```

After (fixed, offset committed after the draft is delivered): the offset is still uncommitted at the kill, the restarted Writer receives the same offset again, finishes it, and only then commits.

```
2026-10-29 19:02:48.533 [harness] started Writer A pid=61956 env={'HW8_MODE': 'fixed'}
2026-10-29 19:02:51.364 [harness] HARD-KILLED Writer A pid=61956
2026-10-29 19:02:51.508 [harness] right after the kill: committed offsets={0: None, 1: None} drafts_for_corr=0
2026-10-29 19:02:51.513 [harness] started Writer A pid=65124 env={'HW8_MODE': 'fixed'}
--- writer 1 (killed) ---
2026-10-29 19:02:48.954 [writer-A] Consumer is ready and listening... topic=s9871_tasks group=s9871_writers_kill_fixed_8c9fb0 mode=fixed idempotent=
      True commit=after poison=dlq
2026-10-29 19:02:49.590 [writer-A] RECEIVED topic=s9871_tasks partition=0 offset=0 corr=kill-fixed-012ba0
2026-10-29 19:02:49.594 [writer-A] WORK start corr=kill-fixed-012ba0
--- writer 2 (restarted) ---
2026-10-29 19:02:59.106 [writer-A] Consumer is ready and listening... topic=s9871_tasks group=s9871_writers_kill_fixed_8c9fb0 mode=fixed idempotent=
      True commit=after poison=dlq
2026-10-29 19:02:59.748 [writer-A] RECEIVED topic=s9871_tasks partition=0 offset=0 corr=kill-fixed-012ba0
2026-10-29 19:02:59.749 [writer-A] WORK start corr=kill-fixed-012ba0
2026-10-29 19:03:07.748 [writer-A] WORK done corr=kill-fixed-012ba0
2026-10-29 19:03:07.748 [writer-A] PROCESSED corr=kill-fixed-012ba0 partition=0 offset=0
2026-10-29 19:03:07.757 [writer-A] SENT topic=s9871_drafts partition=0 offset=0 corr=kill-fixed-012ba0 body={...}
2026-10-29 19:03:07.772 [writer-A] COMMITTED AFTER successful processing offset=0 (next to read = 1)
```

III. Poison message – logs

A message with a valid correlation_id whose `plan` is null (the Writer rejects it every time) is followed by a good message on the same partition.

Before (naive: retry forever): 29 failed attempts in the 15 s the test waited (30 in the saved log by the time the Writer was stopped); the good message behind it never starts (first and last lines shown).

```
2026-10-29 19:03:10.693 [writer-A] RECEIVED topic=s9871_tasks partition=0 offset=0 corr=poison-naive-2cb6b8
2026-10-29 19:03:10.693 [writer-A] ATTEMPT 1/unbounded failed corr=poison-naive-2cb6b8: ValueError: plan must be a non-empty list of non-empty strings
2026-10-29 19:03:11.193 [writer-A] ATTEMPT 2/unbounded failed corr=poison-naive-2cb6b8: ValueError: plan must be a non-empty list of non-empty strings
2026-10-29 19:03:11.694 [writer-A] ATTEMPT 3/unbounded failed corr=poison-naive-2cb6b8: ValueError: plan must be a non-empty list of non-empty strings
```

```
2026-10-29 19:03:12.194 [writer-A] ATTEMPT 4/unbounded failed corr=poison-naive-2cb6b8: ValueError: plan must be a non-empty list of non-empty strings
... (30 ATTEMPT lines in total, no COMMITTED / WORK line for the good message) ...
2026-10-29 19:03:24.752 [writer-A] ATTEMPT 29/unbounded failed corr=poison-naive-2cb6b8: ValueError: plan must be a non-empty list of non-empty strings
2026-10-29 19:03:25.253 [writer-A] ATTEMPT 30/unbounded failed corr=poison-naive-2cb6b8: ValueError: plan must be a non-empty list of non-empty strings
```

After (fixed: 3 attempts, then the dead-letter topic):

```
2026-10-29 19:03:30.096 [writer-A] RECEIVED topic=s9871_tasks partition=0 offset=0 corr=poison-fixed-8a8fed
2026-10-29 19:03:30.096 [writer-A] ATTEMPT 1/3 failed corr=poison-fixed-8a8fed: ValueError: plan must be a non-empty list of non-empty strings
2026-10-29 19:03:30.597 [writer-A] ATTEMPT 2/3 failed corr=poison-fixed-8a8fed: ValueError: plan must be a non-empty list of non-empty strings
2026-10-29 19:03:31.100 [writer-A] ATTEMPT 3/3 failed corr=poison-fixed-8a8fed: ValueError: plan must be a non-empty list of non-empty strings
2026-10-29 19:03:31.104 [writer-A] DEAD-LETTERED corr=poison-fixed-8a8fed after 3 attempts -> topic=s9871_dead_letter partition=0 offset=0 error=
      ValueError: plan must be a non-empty list of non-empty strings
2026-10-29 19:03:31.107 [writer-A] COMMITTED after dead-lettering offset=0 (next to read = 1)
2026-10-29 19:03:31.107 [writer-A] RECEIVED topic=s9871_tasks partition=0 offset=1 corr=good-fixed-eed3c7
2026-10-29 19:03:31.107 [writer-A] WORK start corr=good-fixed-eed3c7
2026-10-29 19:03:31.112 [writer-A] SENT topic=s9871_drafts partition=0 offset=0 corr=good-fixed-eed3c7 body={...}
2026-10-29 19:03:31.121 [writer-A] COMMITTED AFTER successful processing offset=1 (next to read = 2)
```

IV. Ordering – both runs

20 related messages from `VERIFY_SEED` 269871 (4 fixtures, 1 to 8 updates each, saved in `raw/part3_ordering_messages.json`) go through `s9871_tasks` (2 partitions) to two Writers (A and B) in one consumer group. Per entity, the numbers are the update sequence numbers in the order the Writers finished them (1, 2, 3, ... is correct).

Entity	Messages	No key: processed order	No key: partitions / writers	Keyed: processed order	Keyed: partition / writer
FX-9871-101	1	[1]	[1] / ['B']	[1]	[1] / ['B']
FX-9871-102	4	[1, 3, 2, 4]	[0, 1] / ['A', 'B']	[1, 2, 3, 4]	[1] / ['B']
FX-9871-103	8	[2, 1, 6, 7, 3, 4, 5, 8]	[0, 1] / ['A', 'B']	[1, 2, 3, 4, 5, 6, 7, 8]	[1] / ['B']
FX-9871-104	7	[1, 2, 3, 6, 7, 4, 5]	[0, 1] / ['A', 'B']	[1, 2, 3, 4, 5, 6, 7]	[0] / ['A']

Run 1, no key: (processing order, as logged by the two Writers)

```
2026-10-29 19:03:32.842 [harness] assignments: A=[0]; B=[1]
2026-10-29 19:03:32.842 [harness] sending 20 messages, key=none (null key)
processing_order  processed_at  writer  corr  partition  offset  entity_id  entity_seq
1  2026-10-29 19:03:33.347  B  ord-01  1  0  FX-9871-104  1
2  2026-10-29 19:03:33.566  A  ord-02  0  0  FX-9871-102  1
3  2026-10-29 19:03:34.084  A  ord-06  0  1  FX-9871-103  2
4  2026-10-29 19:03:34.167  B  ord-03  1  1  FX-9871-104  2
5  2026-10-29 19:03:34.505  A  ord-08  0  2  FX-9871-104  3
6  2026-10-29 19:03:34.869  A  ord-10  0  3  FX-9871-102  3
7  2026-10-29 19:03:34.993  B  ord-04  1  2  FX-9871-103  1
8  2026-10-29 19:03:35.252  A  ord-15  0  4  FX-9871-103  6
9  2026-10-29 19:03:35.659  B  ord-05  1  3  FX-9871-101  1
10  2026-10-29 19:03:35.725  A  ord-17  0  5  FX-9871-104  6
11  2026-10-29 19:03:35.926  A  ord-18  0  6  FX-9871-104  7
12  2026-10-29 19:03:36.081  B  ord-07  1  4  FX-9871-102  2
13  2026-10-29 19:03:36.559  B  ord-09  1  5  FX-9871-104  4
14  2026-10-29 19:03:36.667  A  ord-19  0  7  FX-9871-103  7
15  2026-10-29 19:03:37.112  B  ord-11  1  6  FX-9871-102  4
16  2026-10-29 19:03:37.443  B  ord-12  1  7  FX-9871-103  3
17  2026-10-29 19:03:37.917  B  ord-13  1  8  FX-9871-103  4
18  2026-10-29 19:03:38.777  B  ord-14  1  9  FX-9871-103  5
19  2026-10-29 19:03:39.397  B  ord-16  1  10  FX-9871-104  5
20  2026-10-29 19:03:39.622  B  ord-20  1  11  FX-9871-103  8
```

Run 2, key = entity_id: (processing order, as logged by the two Writers)

```
2026-10-29 19:03:45.455 [harness] assignments: A=[0]; B=[1]
2026-10-29 19:03:45.455 [harness] sending 20 messages, key=entity_id
processing_order  processed_at  writer  corr  partition  offset  entity_id  entity_seq
1  2026-10-29 19:03:46.013  A  ord-01  0  0  FX-9871-104  1
2  2026-10-29 19:03:46.235  B  ord-02  1  0  FX-9871-102  1
3  2026-10-29 19:03:46.787  A  ord-03  0  1  FX-9871-104  2
4  2026-10-29 19:03:47.017  B  ord-04  1  1  FX-9871-103  1
5  2026-10-29 19:03:47.147  A  ord-08  0  2  FX-9871-104  3
6  2026-10-29 19:03:47.568  A  ord-09  0  3  FX-9871-104  4
7  2026-10-29 19:03:47.665  B  ord-05  1  2  FX-9871-101  1
8  2026-10-29 19:03:48.125  B  ord-06  1  3  FX-9871-103  2
9  2026-10-29 19:03:48.149  A  ord-16  0  4  FX-9871-104  5
10  2026-10-29 19:03:48.495  B  ord-07  1  4  FX-9871-102  2
11  2026-10-29 19:03:48.579  A  ord-17  0  5  FX-9871-104  6
12  2026-10-29 19:03:48.740  A  ord-18  0  6  FX-9871-104  7
13  2026-10-29 19:03:48.816  B  ord-10  1  5  FX-9871-102  3
14  2026-10-29 19:03:49.327  B  ord-11  1  6  FX-9871-102  4
```

```
15  2026-10-29 19:03:49.616  B  ord-12  1  7  FX-9871-103  3
16  2026-10-29 19:03:50.046  B  ord-13  1  8  FX-9871-103  4
17  2026-10-29 19:03:50.836  B  ord-14  1  9  FX-9871-103  5
18  2026-10-29 19:03:51.187  B  ord-15  1  10  FX-9871-103  6
19  2026-10-29 19:03:51.886  B  ord-19  1  11  FX-9871-103  7
20  2026-10-29 19:03:52.068  B  ord-20  1  12  FX-9871-103  8
```

What the numbers show

- **Duplicate delivery.** With no idempotency, replaying one correlation_id made the Writer do the work twice and emit 2 drafts, so everything downstream (Reviewer, final answer) would also run twice. With the processed-correlation_id store the second delivery is logged as a duplicate and produces no output and no side effect. The replay is still read and its offset committed, so lag returns to 0.

- **Consumer death.** Committing the offset *before* the work is at-most-once: the Writer was killed 1.5 s into an 8 s job, the broker already believed the message was handled (committed offset 1, lag 0), and after the restart nothing arrived - the message was lost with no error anywhere. Committing *after* the draft is delivered leaves the offset uncommitted at the kill (none), so the restarted Writer received the same offset again, finished it and only then committed. The price of at-least-once is that work can run twice (the kill happened mid-job), which is exactly why the idempotency store is needed alongside it. One small window remains: a crash between delivering the draft and recording the correlation_id can still produce one duplicate draft; closing it fully needs Kafka transactions, which this exercise does not use.

- **Poison message.** An unprocessable message with unbounded retry stays at the head of its partition forever: 29 attempts in 15 s and the valid message behind it never ran (lag 2). With 3 attempts and a dead-letter topic the bad message is parked with its error, original bytes and source offset, and the next message completed immediately. (The naive poison run keeps commit-after so that only the retry policy differs from the fixed run.)

- **Ordering.** Kafka only orders messages *within a partition.* Without a key the 20 messages were spread over both partitions and two Writers raced, so related messages for the same fixture finished out of order. Keying by entity_id sends every message for a fixture to one partition, hence to one Writer, and the order was preserved for all entities (the keyed run also shows the cost: one Writer handles the busiest fixture on its own).

V. Comparison with the HW2 in-process graph

`code/agent_graph.py` (HW2) is a LangGraph `StateGraph`: Supervisor, Planner and Reviewer are Python functions called one after another in a single process. They share one in-memory `AgentState` dict (including the live model client), the router (`router_logic`) reads that state to decide the next node, a `turn_ceiling` of 6 bounds the loop, and a failed Pydantic validation is retried by routing back to the Planner. Every step waits for the previous function to return.

Two benefits of moving to Kafka

1. *Independent scaling.* In HW2 the only way to do more work is a faster process. Here `s9871_tasks` has 2 partitions and two Writer instances in one consumer group shared the 20 messages (Part 3 IV: Writer A held partition 0, Writer B partition 1) without any change to the Writer code.

2. *Decoupled failure and time.* In HW2 an exception inside any node ends the whole run and the in-memory state is gone. Here the Writer was killed mid-job and the message was still waiting in the topic; the restarted Writer finished it (Part 3 II), and a poison message is parked in `s9871_`

`dead_letter` instead of stopping the pipeline. Planner, Writer and Reviewer can also be stopped and restarted separately because they only share topics.

Two new responsibilities

1. *Duplicate delivery / idempotency.* A function call runs exactly once. Kafka is at-least-once, so a redelivery or replay runs the Writer again; I had to add a durable processed-correlation_id store (and commit offsets only after success) to keep the side effect single.

2. *Ordering.* The HW2 graph's order is decided by its router and is always the same. Kafka guarantees order only inside a partition, so I had to choose a message key (entity_id) to keep related messages in order; anything keyed badly (or unkeyed) is reordered across consumers. Related to this, I now own the retry limit and the dead-letter policy that HW2 handled with `turn_ceiling`.

One honest difference: HW2's Reviewer could send work back to the Planner (feedback loop). The Part 2 pipeline is one-way (Reviewer only approves or rejects), so that self-correction loop is not reproduced here.

Self-check (verification.json)

`verify_hw08.py` starts the real stack and runs 11 objective checks (commit `7cdb88d`, SEED 9871, VERIFY_SEED 269871): 11/11 passed. It was also run against two deliberately broken states and failed where it should (`raw/verify_broken_*.txt`: worker stopped and a file missing -> 4/9 passed; the three fixes switched off -> 8/11 passed, with the duplicate, kill and poison checks failing).

Check	Result
required files exist	PASS
Stack: Kafka + worker + FastAPI running, all topics exist, tasks has 2 partitions	PASS
Part 1: CREATE_ENTITY then GET_ENTITY round-trip through Kafka via the API (uuid, same fields, correlation_id preserved)	PASS
Part 1: clean errors (unknown id, unknown operation, invalid fields) each carry a correlation_id	PASS
Part 1: direct Kafka client - request has correlation_id/reply_to/operation/data, response echoes the correlation_id	PASS
Part 2: Planner, Writer, Reviewer as separate processes (real model) - one question reaches final with the same correlation_id in all four topics	PASS
Part 3 - duplicate replay - 2 drafts without idempotency, 1 with it	PASS
Part 3 - Writer killed mid-work - offset uncommitted, message redelivered after restart, committed only after success	PASS
Part 3 - poison message - 3 attempts, then dead-letter topic, next message still processed	PASS
Part 3 - 20 messages (VERIFY_SEED) keyed by entity stay in one partition and in order	PASS
Ruff passes on the HW8 code	PASS

AI_USE

1. What did you use an AI assistant for, and what did you do yourself?

I used Claude Code mainly as a tutor and debugging assistant. I asked it to explain concepts related to Kafka, Docker Compose, FastAPI, Kafka consumer groups, message ordering, retries, dead-letter queues, and the multi-agent pipeline. I also used it when I was unsure how to begin, encountered an error, or needed help locating or downloading a tool or resource. I wrote and implemented the code, ran the commands and tests on my own machine, reviewed the logs and results, took the screenshots, and decided what to include in the report. I also made the project decisions and completed the Git commits myself.

2. One AI mistake, how it was caught, and how it was fixed

The first version of the worker did not handle `SIGTERM`, so it did not immediately leave its Kafka consumer group when it was stopped. After I rebuilt and restarted the worker container, the old worker remained in the group until its session timed out. Because of this, the new worker did not receive the partition for about 40 seconds. The first three API requests I sent returned a 504 timeout.

3. How did you detect the problem or verify the result?

I tested the API with real `curl` requests after restarting the worker instead of only checking whether the code looked correct. The worker log showed that the partition was assigned about 40 seconds after the container restarted. The three requests were then processed together, which showed that they had not been lost; they had only been delayed.

4. What did you change, and why does it work now?

I added a `SIGTERM` handler that calls `consumer.close()` so the worker leaves the Kafka consumer group immediately. I also changed `session.timeout.ms` to 10 seconds, so a process that is unexpectedly killed is detected more quickly. After restarting the worker again, it was assigned its partition in under one second. This also made the Part 3 worker-kill test run much faster.